

Умный роутинг выплат

Технический отчёт по решению команды «Коломот»
Фёдор Филитович · Артём Корсаков

Репозиторий	github.com/T0vich/kolomot-smart-payout-routing, ветка main
Язык и зависимости	Ruby 3.3, только стандартная библиотека. Внешних гемов нет, bundle install не нужен
Запуск	rake test · rake route · rake validate · rake all
Сдаваемые файлы	routing_decisions_test.json и routing_report_test.json — одна команда rake submit

9

жёстких
ограничений

7

стратегий
распределения

7

профилей роутинга

116

тестов, 776
проверок

0

внешних
зависимостей

Все числа в отчёте получены прогоном кода из репозитория на публичных данных кейса (data/operations_queue_10.json, data/providers.json, data/operations_history.csv) и воспроизводятся командами, указанными под каждой таблицей.

Содержание

- | | |
|--|--|
| 1 Резюме | 8 Объяснимость |
| 2 Задача и в чём её сложность | 9 Состояние провайдеров |
| 3 Архитектура решения | 10 Аналитика и выводы |
| 4 Слой допуска: hard-constraints | 11 Гибкость и расширяемость |
| 5 Слой предпочтения: семь стратегий | 12 Качество реализации |
| 6 Согласование конфликтующих целей | 13 Соответствие критериям оценки |
| 7 Отказ, каскад и fallback | 14 Регламент сдачи и открытые вопросы |

РАЗДЕЛ 1

Резюме

Мы разработали движок распределения выплат между платёжными провайдерами. Для каждой заявки он отбирает провайдеров, которым её вообще допустимо отправить, выбирает лучшего по совокупности семи бизнес-целей, при отказе переходит к следующему, а по итогам прогона объясняет каждое решение и выдаёт рекомендации по настройке роутинга.

Что считаем главным в решении

- Жёсткие ограничения и мягкие цели разведены в два независимых слоя. Стратегия физически не может «уговорить» роутер нарушить лимит: фильтрация допуска всегда идёт до скоринга.
- Семь целей учитываются совместно, а не по очереди — взвешенным композитным скорингом. Веса, пороги и состав правил живут в `config/routing.yml`, а не в коде.
- Конфликт целей разрешается явно и воспроизводимо: порядок политик при ничье задан конфигом, полное равенство разрывается детерминированно, недостижимая цель не блокирует роутинг, а попадает в отчёт с рекомендацией.
- Каждое решение объяснено: в `attempts` лежат все рассмотренные провайдеры с конкретной причиной выбора или исключения и разложением балла по слагаемым.
- Лимиты резервируются в момент выбора провайдера, а не после подтверждения выплаты. Из-за этого наша проверка строго жёстче валидатора организаторов.
- Аналитика не ограничивается процентами: отчёт сам называет причину расхождения с целевым распределением и предлагает конкретный параметр к изменению.

Результат на публичных данных

Проверка	Результат	Команда
Юнит-тесты	116 тестов, 776 проверок, 0 падений	<code>rake test</code>
Валидатор организаторов	29 проверок пройдено, 0 ошибок, 0 предупреждений	<code>rake validate</code>
Публичная очередь	10 из 10 заявок смаршрутизированы, отклонение от целевых долей 10 п.п.	<code>rake route</code>
Backtest по истории	суммарное отклонение от целей 32 → 26 п.п.	<code>rake backtest</code>
CI	зелёный на каждый push и pull request	GitHub Actions

Ограничения кейса соблюдены

Внутри решения нет ни одной нейросети и ни одного проприетарного компонента: весь выбор провайдера — детерминированные правила и арифметика, повторный запуск на тех же данных даёт побайтово тот же результат (тест `test_run_is_reproducible`). Весь код решения — Ruby.

РАЗДЕЛ 2

Задача и в чём её сложность

Формулировка «выбрать наиболее подходящего партнёра» скрывает три разных вопроса. Большинство ошибок в таких системах берётся из того, что их решают одним механизмом.

Вопрос	Свойство ответа	Где решается у нас
Кому вообще можно отправить эту заявку?	Бинарный. Нарушать нельзя никогда, независимо от стратегии	Слой hard-constraints, 9 правил
Кого из допустимых лучше выбрать?	Всегда компромисс между конфликтующими целями	Слой soft-goals, 7 стратегий + композитный скоринг
Что делать, если провайдер отказал ?	Последовательность попыток с сохранением всех жёстких ограничений	Каскад и fallback на self-provider

Конфликт целей — не побочный эффект, а суть задачи

Пример из технического задания: цель — 40 % заявок на vipay; факт — vipay почти исчерпал дневной максимум, а на payflow висит обязательство «не менее 2 000 000 ₽ в сутки». Обе цели корректны, обе одновременно выполнить нельзя. Система обязана не просто выбрать, а объяснить выбор и показать цену компромисса в аналитике.

Это же расхождение видно и на исторических данных кейса — там оно уже стоило денег:

Провайдер	Факт, доля	Цель, доля	Расхождение	Конверсия факт	Конверсия в снимке
vipay	41.0 %	40.0 %	+1.0 п.п.	0.78	0.87
payflow	19.0 %	35.0 %	−16.0 п.п.	0.47	0.91
quickpay	40.0 %	25.0 %	+15.0 п.п.	0.68	0.79

Источник: rake analyze po data/operations_history.csv, 100 операций.

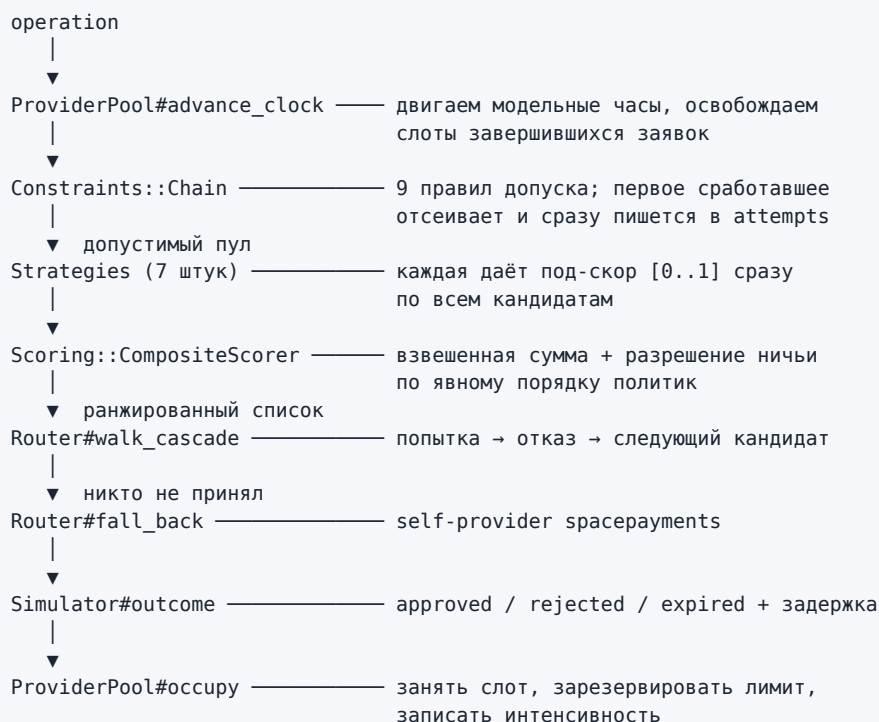
РАЗДЕЛ 3

Архитектура решения

3.1 Два слоя, которые нельзя смешивать

Hard-constraints отвечают на вопрос допуска и применяются до выбора стратегии. Soft-goals работают только внутри уже допустимого пула и влияют на предпочтение, а не на разрешение. Альтернативу — единый скоринг, где нарушение лимита стоит очень большого штрафа, — мы отвергли: в такой схеме нарушение перестаёт быть невозможным и становится просто дорогим, а отладить и объяснить его нельзя.

3.2 Путь одной заявки



3.3 Карта модулей

Модуль	Ответственность
config.rb	Загрузка и валидация routing.yml, профили. Единственное место, где решается «что включено»
provider_pool.rb	Состояние провайдеров, модельное время, агрегаты долей по количеству и объёму
rate_limiter.rb	Скользящее окно 60 секунд для интенсивности
constraints/	9 правил допуска, по классу на правило. Реестр в registry.rb
strategies/	7 мягких целей, по классу на стратегию. Реестр в registry.rb
scoring/	Композитный балл, разрешение конфликта целей, текстовое объяснение выбора
router.rb	Оркестрация: фильтр → скоринг → каскад → fallback → обновление состояния
simulator.rb	Детерминированная симуляция исхода и задержки
analytics/	Калибровка по истории, отчёт, backtest, сравнение профилей, HTML-дашборд
loaders.rb	Чтение входных файлов с внятыми ошибками вместо стектрейса

Модуль	Ответственность
models/	Operation, Provider, Attempt, Decision

3 149 строк Ruby в lib/, 1 212 строк тестов, 471 строка в bin/ и scripts/, 160 строк конфигурации.

РАЗДЕЛ 4

Слой допуска: **hard-constraints**

Все проверки из технического задания реализованы, каждая — отдельный класс с собственным кодом причины отсева. Порядок и состав правил задаётся конфигом: любое можно выключить или переупорядочить, не трогая код.

Правило	Поле провайдера	Код причины в <code>attempts</code>	Класс
Статус провайдера	<code>status</code>	<code>provider_inactive</code>	<code>status.rb</code>
Нижняя граница чека	<code>limit_amount_min</code>	<code>amount_below_minimum</code>	<code>amount_range.rb</code>
Верхняя граница чека	<code>limit_amount_max</code>	<code>amount_exceeds_limit</code>	<code>amount_range.rb</code>
Дневной максимум	<code>daily_amount_limit</code>	<code>daily_limit_exceeded</code>	<code>daily_amount_limit.rb</code>
Лимит одновременных заявок	<code>in_progress_count_limit</code>	<code>in_progress_count_exceeded</code>	<code>in_progress.rb</code>
Лимит суммы в обработке	<code>in_progress_amount_limit</code>	<code>in_progress_amount_exceeded</code>	<code>in_progress.rb</code>
Банковский фильтр	<code>banks / exclude_banks</code>	<code>bank_not_in_list</code> <code>bank_in_exclude_list</code>	<code>bank_filter.rb</code>
Маржа	<code>provider_margin_pct</code> , <code>merchant_margin_pct</code> , <code>allow_negative_agreement</code>	<code>negative_margin</code>	<code>margin.rb</code>
Реквизиты	<code>available_requisites</code>	<code>no_requisites</code>	<code>requisites.rb</code>
Интенсивность	<code>requests_per_minute_limit</code>	<code>rate_limit_exceeded</code>	<code>rate_limit.rb</code>
Потолок оборота	<code>daily_turnover_max</code>	<code>turnover_max_reached</code>	<code>turnover_max.rb</code>

Последнее правило — наше дополнение к списку из ТЗ: финансовое обязательство «не более X ₽ в сутки» может отличаться от технического дневного лимита, и смешивать их нельзя.

Отсев всегда объяснён числами

```
{ "provider": "payflow", "decision": "skipped",
  "reason": "amount_exceeds_limit",
  "details": "150000 > limit_amount_max 50000",
  "stage": "hard_filter" }

{ "provider": "vipay", "decision": "skipped",
  "reason": "bank_not_in_list",
  "details": "raiffeisen отсутствует в banks [sberbank, tinkoff, vtb]",
  "stage": "hard_filter" }
```

РАЗДЕЛ 5

Слой предпочтения: семь стратегий

Реализованы все семь стратегий распределения из технического задания. Каждая — отдельный класс, каждая нормирует свой сигнал в отрезок [0..1] сразу по всем кандидатам, каждая включается весом в профиле. Вес 0 или отсутствие ключа означает, что стратегия в прогоне не участвует.

#	Стратегия	Поле	Как считается под-скор
1	Доля по количеству traffic_share	traffic_percentage	Для каждого кандидата считаем, каким станет суммарное отклонение всех фактических долей от целевых, если заявку отдать именно ему. Меньше отклонение — выше балл
2	Доля по объёму volume_share	volume_share_pct	Та же математика, но доли считаются в рублях: крупная заявка двигает распределение сильнее мелкой
3	Очередь в каскаде cascade_priority	priority	Ранг считается среди доступных кандидатов. Если провайдер с priority 1 отсеян, полный балл получает следующий
4	По сумме чека amount_band	amount_bands (конфиг)	Диапазон влияет на предпочтение , а не на допуск: провайдер вне своего диапазона остаётся допустимым, но получает miss_score вместо match_score
5	По конверсии conversion	conversion_24h + история	$effective = conversion_24h \times (1 - blend) + observed \times blend$, blend = 0.3 по умолчанию
6	По интенсивности load_balance	requests_per_minute_limit и состояние пула	$score = 1 - load$, где load — максимум по узким местам: оборот, in-progress count и amount, реквизиты, заявок в минуту
7	Фин. обязательства turnover_commitment	daily_turnover_min daily_turnover_max	Разрывная шкала: [0.6..1.0] пока минимум не набран, [0.0..0.5] после. Разрыв гарантирует, что невыполненное обязательство всегда весомее его отсутствия

Почему конверсию смешиваем с историей

Заявленная в снимке конверсия расходится с фактической очень сильно — у payflow 0.91 против 0.47 по 100 историческим операциям, то есть на 43.6 процентных пункта. Полностью доверять снимку нельзя; полностью истории — тоже, она за один день. Отсюда параметр history_blend, вынесенный в конфиг.

Нормировка: сравнительная и абсолютная

Часть сигналов имеет смысл только в сравнении с текущими кандидатами, часть — сама по себе. traffic_share, volume_share, conversion и cascade_priority нормируются min-max по пулу: иначе разница конверсий 0.79 против 0.91 растворяется среди остальных слагаемых. load_balance, turnover_commitment и amount_band считаются по своей абсолютной шкале — «загружен на 64 %» осмысленно без сравнения. Следствие, которое стоит знать: при двух кандидатах сравнительные стратегии дают ровно 1.0 и 0.0, и выбор превращается во взвешенное голосование.

РАЗДЕЛ 6

Согласование конфликтующих целей

Техническое задание требует, чтобы факторы учитывались совместно, а не по очереди, и чтобы было определено поведение в трёх ситуациях: цели указывают на разных провайдеров, цели равны, цель невыполнима. Все три случая закрыты явно.

6.1 Совместный учёт: композитный балл

$$\text{score}(p) = \sum (\text{под-скор}_i(p) \times \text{вес}_i) / \sum \text{вес}_i$$

Весы профиля `balanced`, который сдаётся по умолчанию:

traffic_share	volume_share	conversion	turnover_commitment	cascade_priority	amount_band	load_balance
0.26	0.16	0.18	0.14	0.10	0.09	0.07

Весы заданы в `config/routing.yml`. Смена приоритетов бизнеса — правка конфига, а не кода: профиль `conversion_first` уже лежит в репозитории и переключается флагом `--profile`.

6.2 Цели указывают на разных провайдеров — и разрыв мал

Если разница баллов лидеров не превышает `tie_break_epsilon`, применяется явный порядок политик `conflict_order`:

фин. обязательства → доля по количеству → конверсия → доля по объёму
→ каскад → диапазон суммы → загрузка

Обязательство по обороту стоит выше целевой доли осознанно: за недобор минимального оборота платят деньгами по договору, а доля — ориентир, который можно добрать следующими заявками. Порядок задаётся конфигом и валидируется при загрузке: неизвестная стратегия в `conflict_order` — ошибка старта, а не тихое игнорирование.

6.3 Цели равны полностью

Детерминированный разрыв: сначала `priority`, затем имя провайдера. Один и тот же вход всегда даёт один и тот же выход — это проверяется тестом `test_ranking_is_deterministic_for_identical_candidates`.

6.4 Цель невыполнима

Требуемая доля может относиться к провайдеру, которого прямо сейчас отсекают `hard-constraints`. Такая цель не блокирует роутинг: заявка уходит следующему кандидату, а расхождение фиксируется в отчёте с указанием причины и конкретной рекомендацией. На публичных данных это видно буквально:

Живая рекомендация из `routing_report.json`

«viraу отсеян 4 раза по фильтру банка — расширить список `banks` или подключить недостающие банки, иначе целевая доля недостижима.»

6.5 Что это даёт на конкретной заявке

ор_101, 15 000 ₽, Сбербанк. Допустимы все три провайдера, и цели расходятся: по диапазону суммы предпочтителен `rauflow`, по загрузке — `quickrau`, по долям и конверсии — `viraу`. Скоринг сводит это в одно число.

Слагаемое	vipay	payflow	quickpay
Доля по количеству × 0.26	1.00 → 0.26	0.67 → 0.17	0.00 → 0.00
Доля по объёму × 0.16	1.00 → 0.16	0.25 → 0.04	0.00 → 0.00
Конверсия × 0.18	1.00 → 0.18	0.27 → 0.05	0.00 → 0.00
Фин. обязательства × 0.14	0.18 → 0.03	0.50 → 0.07	0.50 → 0.07
Каскад × 0.10	1.00 → 0.10	0.50 → 0.05	0.00 → 0.00
Диапазон суммы × 0.09	0.25 → 0.02	1.00 → 0.09	0.25 → 0.02
Загрузка × 0.07	0.36 → 0.03	0.03 → 0.00	0.86 → 0.06
Итоговый балл	0.77	0.47	0.15

РАЗДЕЛ 7

Отказ, каскад и fallback

7.1 Два разных события, названные в ТЗ одним словом

Событие	Что значит	Что делает роутер
Провайдер не принял заявку	Ретраибельный отказ: заявка в работу не ушла	Исключает провайдера из пула и переходит к следующему кандидату. Кандидаты кончились — self-provider spacepayments
Провайдер принял и обработал	Терминальный исход: approved, rejected или expired	Маршрут не меняется, исход пишется в simulated_result

Разделение принципиальное. Если считать неуспешную выплату отказом и перемаршрутизировать, то безальтернативная заявка вроде op_103 уходит в spacepayments — а эталонный файл организаторов reference_decisions.json требует для неё строго quickpay. Поэтому искусственные отказы первого рода в сдаваемом профиле выключены: прогон должен быть воспроизводимым. Механика каскада от этого не исчезает — она включается профилем demo_failover и покрыта тестами.

Вопрос к жюри, готовый к переключению

Если организаторы подтвердят, что неуспешная выплата требует перемаршрутизации, поведение включается флагом `retry_on_terminal_failure` в `config/routing.yml`, сдаваемые файлы регенерируются одной командой `rake submit`. Изменений в коде не нужно.

7.2 Каскад в действии

Профиль `demo_failover`, команда `rake demo`. Заявка `op_103` на 150 000 ₽:

```

vipay      skipped  amount_exceeds_limit  150000 > limit_amount_max 100000
payflow    skipped  amount_exceeds_limit  150000 > limit_amount_max 50000
quickpay    skipped  provider_declined      провайдер отказался принять заявку,
                                     переходим к следующему кандидату
spacepayments selected fallback_self_provider  внешних допустимых провайдеров
                                     не осталось, заявка ушла на self-provider

```

Сводка каскада попадает в отчёт отдельным разделом:

Показатель	Сдаваемый профиль balanced	Профиль demo_failover
Заявок с повторной попыткой	0	8
Всего повторных попыток	0	8
Ушло на self-provider	0	4
Провайдеров рассмотрено на заявку	3.0	3.4

7.3 Fallback не нарушает жёсткие ограничения

Переход к следующему кандидату идёт по уже отфильтрованному пулу: провайдер, не прошедший `hard-constraints`, не может стать запасным вариантом ни при каких обстоятельствах. Число попыток ограничено параметром `max_attempts`. Поведение проверяется тестами `test_cascade_moves_to_next_provider_on_decline`, `test_falls_back_to_self_provider_when_all_decline`, `test_falls_back_when_pool_is_empty_by_hard_constraints` и `test_respects_max_attempts`.

РАЗДЕЛ 8

Объяснимость

Требование T3 — сохранять понятное объяснение: какие варианты рассматривались, почему выбран конкретный партнёр и по каким причинам исключены остальные. В нашем формате `attempts` содержит всех рассмотренных провайдеров в порядке рассмотрения, у каждого — стадия, код причины, человекочитаемая расшифровка, а у прошедших фильтр ещё и балл с разложением по слагаемым.

8.1 Почему выбран именно он

```
{ "provider": "vipay",
  "decision": "selected",
  "reason": "best_composite_score",
  "details": "балл 0.77 против 0.47 у payflow; основной вклад — «Доля по количеству заявок». доля по количеству 0.0% при цели 40.0% (недобирает 40.0 п.п.); доля по объёму 0.0% при цели 45.0%; conversion_24h 0.87, по истории 0.78 (смешано 0.3); оборот 32000000 ₺ при потолке 50000000 ₺/сутки; priority 1 в каскаде; загрузка 64.0% по узкому месту «дневной оборот»",
  "stage": "routed",
  "score": 0.77,
  "score_breakdown": { "traffic_share": { "score": 1.0, "weight": 0.26, "contribution": 0.26 }, ... } }
```

Объяснение всегда называет ближайшего конкурента и его балл — иначе «0.77» ничего не значит. Для безальтернативной заявки причина другая и точная: `only_eligible_provider`, «единственный провайдер, прошедший `hard-constraints`».

8.2 Почему исключены остальные

Стадия	Коды причин	Что значит
hard_filter	amount_exceeds_limit, amount_below_minimum, bank_not_in_list, bank_in_exclude_list, daily_limit_exceeded, in_progress_count_exceeded, in_progress_amount_exceeded, no_requisites, negative_margin, rate_limit_exceeded, provider_inactive, turnover_max_reached	Провайдер недопустим. details содержит конкретные числа сравнения
scoring	lower_score	Провайдер допустим, но проиграл по совокупному баллу. details называет его балл
cascade	provider_declined	Провайдер не принял заявку, роутер пошёл дальше

Разделение стадий важно для аналитики: в отчёте `skip_reasons` содержит только жёсткие отсевы, а проигрыши по баллу вынесены в отдельный раздел `soft_skip_reasons`. Иначе статистика причин отсева смешивает «нельзя» и «можно, но не лучший» и становится бесполезной.

8.3 Полнота цепочки

Тест `test_every_eligible_provider_is_explained` требует, чтобы каждый провайдер, прошедший `hard-constraints`, получил запись в `attempts` — либо `selected`, либо `lower_score`. Тест `test_exactly_one_selected_attempt_per_operation` гарантирует, что выбранный ровно один и он совпадает с `selected_provider`.

РАЗДЕЛ 9

Состояние провайдеров

9.1 Резервирование лимитов вместо учёта пост-фактум

Наивная реализация увеличивает `daily_approved_amount` только после подтверждения выплаты. Тогда десять заявок, отправленных подряд, все проходят проверку дневного лимита — и лимит оказывается продан несколько раз. Мы резервируем сумму в момент выбора:

```
занятый лимит = daily_approved_amount + daily_reserved_amount
```

Когда заявка завершается, резерв снимается: при `approved` сумма переезжает в `daily_approved_amount`, иначе просто освобождается. Тот же принцип для `in_progress_count` и `in_progress_amount`.

Побочная выгода, важная для проверки

Валидатор организаторов считает лимиты по исходному снимку `providers.json`, как будто ни одной заявки ещё не отправляли. Наша проверка строго жёстче — значит, выбранный нами провайдер гарантированно входит в его список допустимых. На публичной очереди это дало 29 пройденных проверок из 29 без единого предупреждения.

9.2 Модельное время

Заявка в обработке занимает слот не навсегда: она держит его `latency_sec` и освобождает. Роутер двигает часы по `created_at` очередной заявки и списывает всё, что успело завершиться. На короткой публичной очереди разница незаметна, на длинной тестовой — решает, упрётся провайдер в лимит одновременных заявок или нет.

9.3 Интенсивность

`RateLimiter` хранит отметки времени отправленных заявок в скользящем окне 60 секунд. Он используется дважды: как жёсткое ограничение (превышение `requests_per_minute_limit` исключает провайдера) и как сигнал для мягкой стратегии `load_balance` (текущая загрузка снижает предпочтение задолго до достижения потолка).

9.4 Что обновляется после каждой заявки

Величина	Когда занимается	Когда освобождается
<code>daily_reserved_amount</code>	при выборе провайдера	при завершении заявки
<code>daily_approved_amount</code>	при завершении с исходом <code>approved</code>	—
<code>in_progress_count</code>	при выборе провайдера	по истечении <code>latency_sec</code>
<code>in_progress_amount</code>	при выборе провайдера	по истечении <code>latency_sec</code>
<code>available_requisites</code>	при выборе провайдера	по истечении <code>latency_sec</code>
окно интенсивности	при выборе провайдера	выпадает из окна через 60 с

РАЗДЕЛ 10

Аналитика и выводы

routing_report.json содержит обязательные разделы из ТЗ и ещё шесть наших. Дополнительные поля разрешены условиями кейса.

Раздел отчёта	Что показывает	Источник
period, total_operations	Период и объём прогона	ТЗ
distribution	Доля по количеству и по объёму, цель, отклонение, конверсия — по каждому провайдеру	ТЗ, расширено
skip_reasons	Статистика жёстких отсевов	ТЗ
projected_daily_utilization	Использование дневного лимита	ТЗ
recommendations	Что конкретно поменять в настройках	ТЗ
routing_profile	Имя профиля, веса, порядок разрешения конфликтов	наше
results	approved / rejected / expired, успешность, средняя задержка	наше
soft_skip_reasons	Проигрыши по баллу, отдельно от жёстких отсевов	наше
limits_pressure	Давление на все лимиты, а не только на дневной	наше
cascade	Повторные попытки, использование fallback	наше
deviations	Причина каждого существенного расхождения с целью (порог 10 п.п.)	наше
deviations_note	Что считается существенным, а если таких нет — почему остаток неустраним	наше
history_calibration	Факт по истории и дрейф заявленной конверсии	наше

10.1 Распределение на публичной очереди

Провайдер	Заявок	Факт	Цель	Откл.	Объём, ₽	Объём факт	Объём цель
vipay	4	40.0 %	40.0 %	0.0	102 000	26.4 %	45.0 %
payflow	3	30.0 %	35.0 %	−5.0	88 800	23.0 %	30.0 %
quickpay	3	30.0 %	25.0 %	+5.0	195 000	50.5 %	25.0 %

Отклонение по количеству не случайно и минимально возможно: 4 из 10 заявок безальтернативны по hard-constraints. op_103 (150 000 ₽ при потолке 100 000 у vipay и 50 000 у payflow), op_104 и op_108 (Газпромбанк и Райффайзен отсутствуют в списках банков у vipay и payflow) обязаны уйти в quickpay, op_107 (800 ₽) — в payflow. При цели 25 % и десяти заявках ближе, чем на 5 п.п., к цели подойти арифметически нельзя.

Пустой список отклонений — тоже вывод, и отчёт его формулирует

Существенным считается расхождение от 10 п.п.; такие попадают в deviations с причиной. Когда таких нет, отчёт не молчит: поле deviations_note на этой очереди сообщает — «существенных отклонений нет: наибольшее 5.0 п.п. при пороге 10.0 п.п.; в очереди 10 заявок, поэтому фактическая доля кратна 10.0 п.п., и отклонения меньше этого шага вызваны дискретностью очереди, а не политикой роутинга».

10.2 Давление на лимиты

Провайдер	Использовано, ₽	Дневной лимит, ₽	Утилизация	In-progress count	Оборот min
vipay	3 292 000	5 000 000	65.8 %	40.0 %	нет
payflow	2 948 800	3 000 000	98.3 %	25.0 %	2 000 000 ₽ — выполнен
quickpay	1 295 000	8 000 000	16.2 %	6.7 %	нет

10.3 Дрейф заявленной конверсии — главная находка по данным

Провайдер	Заявлено в снимке	Факт по 100 операциям	Дрейф
vipay	0.87	0.78	−8.95 п.п.
payflow	0.91	0.47	−43.63 п.п.
quickpay	0.79	0.68	−11.50 п.п.

Провайдер с самой высокой заявленной конверсией на деле худший. Роутинг, который берёт `conversion_24h` как есть, будет систематически отправлять заявки туда, где они чаще всего не проходят. Поэтому конверсия у нас смешанная, а дрейф вынесен в отчёт отдельным разделом и в рекомендации.

10.4 Рекомендации: конкретный параметр, а не «стоит улучшить»

Все восемь рекомендаций сгенерированы автоматически на публичной очереди. Каждая называет провайдера, число и параметр, который предлагается изменить.

- payflow выбрал 98.29 % дневного лимита — поднять `daily_amount_limit` до 3 600 000 ₽ или снизить его долю трафика, иначе заявки начнут отсеиваться.
- vipay отсеян 4 раза по фильтру банка — расширить список `banks` или подключить недостающие банки, иначе целевая доля недостижима.
- 2 отсева по нижней границе суммы — пересмотреть `limit_amount_min`: мелкие чеки сейчас упираются в одного исполнителя.
- 3 отсева по верхней границе суммы — крупные чеки уходят к одному провайдеру, стоит поднять `limit_amount_max` у альтернативы.
- payflow: заявленная конверсия 0.91, по истории 0.47 (−43.63 п.п.) — обновить `conversion_24h` или увеличить `history_blend` в профиле.

10.5 Backtest: проверка на 100 исторических операциях

Мы переиграли историю через наш роутер. Метрика выбрана так, чтобы не быть замкнутой на себя: ожидаемая конверсия считается по фактической успешности провайдеров из той же истории, а не по нашему симулятору.

Провайдер	Цель	Было в истории	Стало у нас
payflow	35.0 %	19.0 %	25.0 %
quickpay	25.0 %	40.0 %	38.0 %
vipay	40.0 %	41.0 %	37.0 %
Суммарное отклонение	—	32.0 п.п.	26.0 п.п.

Отдельная находка backtest

29 из 100 исторических выплат ушли провайдеру, который в тот момент по правилам допуска из ТЗ вообще не мог их принять. Это не оценка нашего роутера, а характеристика данных — и лучший аргумент в пользу того, что слой `hard-constraints` нужен отдельно от стратегий.

10.6 Сравнение политик — цена выбора стратегии

Профиль	Отклонение от целей	Ожидаемая конверсия
balanced (сдаваемый)	26.0 п.п.	0.66
traffic, volume, commitments	26.0 п.п.	0.66
cascade	38.0 п.п.	0.69
conversion_first	38.0 п.п.	0.69

Это и есть конфликт целей в числах: соблюсти доли или максимизировать успешность. Три процентных пункта конверсии стоят двенадцати пунктов отклонения от целевых долей. Решение не выбирает за бизнес — оно даёт переключатель и показывает цену. Команда: `rake compare BACKTEST=1`.

РАЗДЕЛ 11

Гибкость и расширяемость

Что нужно изменить	Что делаем	Строк кода
Добавить провайдера	Строка в providers.json	0
Изменить лимиты, банки, конверсию	Правка providers.json	0
Изменить веса целей	Правка профиля в config/routing.yml	0
Изменить порядок разрешения конфликтов	Ключ conflict_order в конфиге	0
Выключить любое правило допуска	Ключ в разделе constraints конфига	0
Другая политика роутинга	Новый профиль + флаг --profile	0
Новое правило допуска	Класс в constraints/ + строка в Registry::ALL + ключ в конфиге	~25
Новая стратегия	Класс в strategies/ + строка в Registry::ALL + вес в профиле	~30

Профили, уже лежащие в репозитории

Профиль	Политика
balanced	Все семь целей вместе. Сдаётся по умолчанию
cascade	Чистый последовательный обход по priority, как депозиты в проде
traffic	Только доля по количеству заявок
volume	Доля по объёму 75 % + доля по количеству 25 %
conversion_first	Максимизация успешности, доли вторичны
commitments	Приоритет финансовых обязательств по обороту
demo_failover	Демонстрация каскада и fallback: искусственные отказы включены

Решение не привязано к входным данным кейса

Ни одно имя провайдера не зашито в код. Список банков, диапазоны сумм, лимиты, целевые доли, приоритеты, дополнительные поля (volume_share_pct, priority, requests_per_minute_limit, daily_turnover_min, daily_turnover_max) читаются из данных и конфигурации. Дополнительные поля можно держать и в providers.json, и в конфиге — код читает оба источника, конфиг имеет приоритет. Это сделано специально: организаторы могут выдать вместе с тестовой очередью свой providers.json.

РАЗДЕЛ 12

Качество реализации

12.1 Тесты

Файл	Тестов	Что покрывает
constraints_test.rb	16	Каждое правило допуска, включая nil-лимиты, чёрные списки банков, скользящее окно, выключенное правило
strategies_test.rb	12	Каждая стратегия по отдельности: реакция на голодающего провайдера, на сумму, на загрузку, смешивание с историей
router_test.rb	13	Каскад, fallback, обновление состояния, освобождение слотов, max_attempts, сходимость к целевым долям
pipeline_test.rb	12	Сквозной прогон: совпадение с эталоном организаторов, обязательные поля, отсутствие нарушений лимитов, воспроизводимость, все профили
loaders_test.rb	11	Битый JSON, отсутствующий файл, дубли operation_id, amount строкой, нулём, отрицательный, провайдер без обязательного поля
report_test.rb	16	Обязательные ключи из ТЗ, сходимость долей к 100 %, разделение жёстких и мягких причин, пустая очередь
config_test.rb	9	Неизвестный профиль, неизвестная стратегия, пустые веса, невалидный conflict_order, отсутствующий файл
analytics_test.rb	15	CSV с кавычками и запятыми, пустая история, backtest, сравнение профилей, экранирование HTML в дашборде
scoring_test.rb	6	Взвешенное среднее, нулевые веса, разрешение ничьи, детерминизм, текст объяснения
simulator_test.rb	6	Детерминизм при одном seed, границы конверсии, положительная задержка, отказы по профилю
Итого	116	776 проверок, 0 падений, 0 пропусков

12.2 Обработка ошибок и граничных ситуаций

Ситуация	Поведение
Файл не найден, битый JSON	Ошибка с путём и причиной разбора, а не стектрейс
Заявка без id, с amount строкой, нулём, отрицательным	Отклоняется на загрузке с указанием поля и значения
Дубли operation_id	Отклоняются на загрузке: молча смаршрутизировать дубль хуже, чем упасть
Очередь из одного объекта вместо массива	Принимается: валидатор организаторов допускает оба варианта
Пустая очередь	Валидный отчёт с нулями, без деления на ноль
providers.json не массив, пустой, без обязательного поля	Ошибка с указанием, чего именно не хватает
Неизвестная стратегия или правило в конфиге	Ошибка при старте, а не тихое игнорирование
Заявка, под которую не подходит никто	Уходит на self-provider spacepayments, помечается fallback_used
daily_amount_limit = null	Трактуется как отсутствие лимита, а не как ноль

Ситуация	Поведение
Провайдер закончился по реквизитам посреди очереди	Покидает пул, остальные заявки идут дальше

12.3 Непрерывная интеграция

GitHub Actions на каждый push в main и каждый pull request: тесты, прогон публичной очереди, валидатор организаторов и структурная проверка сдаваемых файлов, как только они появятся в корне. Ветка main защищена: требуется зелёный чек, force-push и удаление ветки запрещены. Сломанный main на этом кейсе стоит 40 баллов, поэтому проверка автоматическая, а не «посмотрели глазами».

РАЗДЕЛ 13

Соответствие критериям оценки

Технические критерии жюри, 140 баллов. Для каждого подкритерия — где именно в решении он закрыт.

Критерий	Б.	Где в решении
Hard-constraints при определении доступных провайдеров	7	Слой constraints/, 9 классов и 11 кодов причин, chain применяется до скоринга. Раздел 4
Soft-goals при выборе провайдера	7	Слой strategies/, 7 классов, композитный скоринг. Разделы 5–6
Корректное обновление состояния после операций	5	ProviderPool: резервирование и освобождение лимитов, модельное время, скользящее окно. Раздел 9
Следующая попытка при отказе и fallback	3	Router#walk_cascade и #fall_back, профиль demo_failover, 4 теста. Раздел 7
Распределение по количеству операций	3	Стратегия traffic_share
Распределение по объёму платежей	3	Стратегия volume_share
Распределение по приоритету провайдера	3	Стратегия cascade_priority, ранг среди доступных
Диапазон суммы влияет на выбор, а не только ограничивает	3	Стратегия amount_band отдельно от жёсткого правила amount_range
Учёт успешности и конверсии	3	Стратегия conversion со смешиванием заявленной и фактической
Загрузка влияет на выбор, а не только проверяется	3	Стратегия load_balance отдельно от жёсткого правила rate_limit
Заданная доля трафика и другие бизнес-параметры	3	Стратегия turnover_commitment: минимальный и максимальный оборот
Параметры правил меняются через конфигурацию	6	config/routing.yml: веса, пороги, состав правил, порядок конфликтов, диапазоны, профили
Новые правила и провайдеры без переработки архитектуры	5	Реестры constraints и strategies, providers.json без правок кода. Раздел 11
Формализованный механизм совместного учёта факторов	7	Взвешенный композитный балл, веса в конфиге. Раздел 6.1
Правило выбора при расхождении целевых факторов	4	tie_break_epsilon + conflict_order, детерминированный разрыв полного равенства. Разделы 6.2–6.3
Поведение при невыполнимой цели	4	Цель не блокирует роутинг, расхождение уходит в deviations и recommendations. Раздел 6.4
Конкретная причина выбора провайдера	4	reason + details + score_breakdown с вкладом каждого слагаемого. Раздел 8.1
Конкретные причины исключения остальных	4	Отдельный код причины на каждое правило, числа в details. Раздел 8.2
Сохранена последовательность рассмотрения и повторов	2	attempts в порядке рассмотрения, поле stage, сводка cascade в отчёте
Фактическая доля и её отклонение от целевой	4	distribution: доли по количеству и объёму, цель, отклонение
Успешность, отказы, загрузка, использование лимитов	3	results, skip_reasons, soft_skip_reasons, projected_daily_utilization, limits_pressure

Критерий	Б.	Где в решении
Причины существенных отклонений	2	deviations с причиной каждого; deviations_note объясняет остаток, когда существенных нет. Раздел 10.1
Рекомендации с конкретным параметром	2	recommendations: провайдер, число, параметр к изменению. Раздел 10.4
Архитектура разделяет компоненты	4	Разделы 3.1 и 3.3
Понятны логика и порядок запуска	3	README, четыре документа в docs/, rake-задачи под каждый сценарий
Обработка ошибок и граничных ситуаций	3	Раздел 12.2, 11 тестов загрузки и 9 конфигурации
routing_decisions_test.json в корне main	20	rake submit, регламент сдачи в разделе 14
routing_report_test.json в корне main	20	rake submit, регламент сдачи в разделе 14

РАЗДЕЛ 14

Регламент сдачи и открытые вопросы

14.1 Формирование сдаваемых файлов

За час до стопкода организаторы выдают `operations_queue_test.json`. Оба требуемых файла генерируются одной командой, без ручных шагов:

```
ср <выданный файл> data/operations_queue_test.json
rake submit
# → routing_decisions_test.json   в корне репозитория
# → routing_report_test.json       в корне репозитория
```

Контрольный список перед пушем в `main`:

1. Количество решений совпадает с количеством заявок в выданной очереди.
2. `operation_id` взяты из новой очереди, а не из старой.
3. `rake validate FILE=routing_decisions_test.json` проходит без ошибок.
4. Оба файла лежат в корне репозитория, а не в подпапке.
5. Файлы попали в ветку `main`, CI зелёный, оба файла видны на GitHub в браузере.

Неправильное имя, не тот каталог или не та ветка = 0 баллов из 40. Проверка делается вдвоём по списку. Если организаторы выдадут вместе с очередью свой `providers.json`, путь к нему передаётся флагом `--providers`, менять код не нужно.

14.2 Вопросы к жюри и готовность переключиться

Часть трактовок ТЗ допускает два прочтения. По каждой мы выбрали более строгий вариант и заранее подготовили переключатель — если ответ жюри окажется другим, меняется конфиг, а не код.

Вопрос	Наша трактовка	Что переключаем, если ответ другой
Неуспешная выплата — отказ, требующий перемаршрутизации, или финал?	Финал. К следующему провайдеру переходим, только если он не принял заявку в работу	Флаг <code>retry_on_terminal_failure</code> в конфиге, затем <code>rake submit</code>
Скрытая проверка считает лимиты по исходному снимку <code>providers.json</code> ?	Да, поэтому наша проверка намеренно строже: резервируем лимит при выборе	Ослабить резервирование в конфиге; менее строгий вариант нам не нужен
Дополнительные поля провайдера — в конфиг или в <code>providers.json</code> ?	Держим в конфиге, читаем оба источника	Ничего: <code>providers.json</code> уже читается
Можно ли добавлять свои поля в решение?	Да, ТЗ прямо разрешает дополнительные поля в отчёте	Убрать <code>score</code> , <code>score_breakdown</code> , <code>stage</code> , <code>amount</code> , <code>bank</code> , <code>eligible_providers</code> из <code>Models::Decision#to_h</code> и <code>Models::Attempt#to_h</code>
Показывать ли <code>spaserayments</code> в распределении отчёта?	Попадает, только если получил хотя бы одну заявку	Ключ в конфиге отчёта
Что писать в поле <code>period</code> ?	Дату из <code>snapshot_at</code> в <code>providers.json</code> , иначе самую раннюю дату из заявок	Флаг <code>--period</code>

14.3 Соблюдение ограничений кейса

- Нейросети внутри проекта не используются: выбор провайдера — детерминированные правила и арифметика. Повторный запуск даёт тот же результат, это закреплено тестом.
- Проприетарных решений и компонентов с закрытым исходным кодом нет: только стандартная библиотека Ruby 3.3.
- Весь код решения написан на Ruby, включая генератор HTML-дашборда и валидацию входных данных.

Отчёт подготовлен командой «Коломот». Все числовые значения получены прогоном кода из ветки main на публичных данных кейса и воспроизводятся командами `rake test`, `rake route`, `rake validate`, `rake analyze`, `rake backtest`, `rake compare`.